Exponential time hypothesis is false... but the verified algorithm is slower than trying all 2^n assignments at every practical size.

Alex Linhares
Argo Lab AGI; on leave from the Getulio Vargas Foundation
alex@argolab.org ORCID 0000-0002-4227-6879

November 5, 2026

Abstract

Placeholder. Item 09 writes the abstract (under 200 words; no number without a section that regenerates it). The title's sentence made precise: what is verified is a guarantee with existential constants and the exponent $n(\log\log n)^2/\sqrt{\log n}$, whose factor exceeds 1 for every $50 \le n < e^{5504}$, so the verified guarantee is weaker than the trivial $2^n \cdot \text{poly}$ guarantee at every practical size; the machine's running time is unknown because the constants are never computed; the manuscript's sharper exponents are unverified; the statement does refute ETH; the audit found no flaw.

1 Introduction

Placeholder. Item 09 drafts this section: the title's sentence made precise in the first paragraph (a guarantee, not a running time; existential constants never computed; $\rho(n) > n$ for every $50 \le n < e^{5504}$; the sharper exponents unverified); the one-paragraph summary; what is claimed and what is not; the stance (a careful reading by people who checked; no flaw found). Ends with what this does not claim".

2 The verified statement, read exactly

The whole of what is machine-checked about family 378 is one theorem, `exact_count_and_sat`, stated in the 220-line comparator file `lean/ComparatorChallenges/SubexponentialSAT.lean` of the collection (OpenAI, 2026) at commit `4eaee875`. The file carries the statement with a `sorry` for its proof (line 218); the solution tree proves the same statement, and the collection's comparator checks that the two agree (Section 4). The replay of Section 5 rebuilt that proof and found the three standard axioms and nothing else. This section reads the statement, definition by definition, and says what it asserts and what it does not. Line numbers are those of the comparator file; we quote nothing from memory.

2.1 The definitions, one by one

Formulas and the count (lines 13–51). A `Literal V` is a variable with a sign (lines 19–22); a `Clause V` is a `Finset` of literals and a `CNF V` a `Finset` of clauses (lines 24–25). An assignment is any function $V \to$ `Bool` (line 17). A literal evaluates to the Boolean "the assignment agrees with its sign" (lines 29–30), a clause to "some literal is true" (lines 32–33), a formula to "every clause is true" (lines 35–36). `WidthAtMost k F` says every clause has at most k literals as a set (line 38). Over a finite variable type, `satisfyingAssignments F` filters the universe of all assignments (lines 44–45), and `count F` is its cardinality (line 47): the number of satisfying assignments among all 2^n when $V =$ `Fin` n. The `Finset` choices change nothing in the meaning: a repeated clause or literal collapses, the empty clause is allowed and unsatisfiable, a tautological clause $\{x, \neg x\}$ is allowed and counts two toward the width. That is consistent with the manuscript's "Repeating a literal permits clauses with fewer than k literals" (p. 1, line 44).

The input encoding (lines 53–81). `encodeNat n` is n ones followed by a zero (line 55): unary. `frame` doubles a bit string, prefixing each bit with a one, and ends it with a zero (lines 57–58). `encodeList enc` prefixes each element's frame with a one and ends the list with a zero (lines 60–62). A literal becomes the number $2 \cdot \text{var} + [\text{positive}]$, at most $2n - 1$ (lines 64–65); a clause is its sorted list of literal numbers, a formula its sorted list of clauses (lines 67–71). `encodeCNF F` is `encodeNat n` followed by the formula's list of lists of unary numbers (lines 73–77), and `formulaBitLength F` is the length of that bit string (line 79). Two things follow. The number of variables n is part of the input, so a variable that occurs in no clause is still counted. And the length is polynomial: a clause of $j \le k$ literals costs $2j(4n+1)+4$ bits, so

$$L \;=\; \texttt{formulaBitLength}\, F \;\le\; n + 2 + m\left(2k(4n+1)+4\right) \;=\; O(k\,n\,m)$$

for m clauses. The solution itself proves $(L+1)^d \le (8k+7)^d\,(n+|F|+1)^{2d}$ (`SqrtLogSpace/Statement.lean`, lines 9–14). Emulating the definitions (`reports/scratch/eth_audit/check_encoding.py`), a formula with $n = 125$, $m = 500$, $k = 4$ encodes to $L =$ 1,006,223 bits and one with $n =$ 1,000, $m =$ 4,000, $k = 3$ to 47.9 million, both under the bound. Nothing in the encoding is exponential; the unary numbers are linear in n.

The machine (lines 83–128). `PrimitiveMachine` (lines 92–99) is a finite object: a number `tapeCount` of tapes and a number `stateCount` of states, an input tape and a start state, a table `scan` saying which tape each state reads, and a table `transition` from `Fin stateCount` $\times$ `Option Bool`—a domain of $3 \cdot$ `stateCount` points—into either a halting answer (a `Bool`) or a next state with one action. The action type is Mathlib's `Turing.TM0.Stmt (Option Bool)`: move the head left or right, or write one of the three symbols `none`, `some false`, `some true`, five values in all (`Mathlib/Computability/TuringMachine/PostTuringMachine.lean` lines 118–120 and `Tape.lean` lines 421–423 at Mathlib `d13f23b7`, the clone's pinned revision). A configuration `Cfg` (lines 103–105) is a state, halted or not, and one Mathlib `Tape` per tape index (a head symbol and two blank-padded halves, `Tape.lean` lines 408–414, blank = `none`). `step` (lines 107–119) returns `none` in a halted state; otherwise it reads the head of the one tape the state scans, looks the pair up in the table, and either halts with the answer, or writes one symbol or moves that one head by one cell and changes state. `init` (lines 121–124) puts the input on the input tape with the head at its first bit and every other tape blank (`Tape.mk`$_1$, `Tape.lean` line

494). A Lean function on a finite domain is, whatever term defines it, a finite table; so a step is one lookup in a table of $3 \cdot$ `stateCount` rows, and no field of the structure can carry unbounded computation.

Traces (lines 87–90). `Trace step t a c` is an inductive proposition with two constructors: `refl`, a trace of length 0 from a to a itself, and `cons`, which extends a trace from b by one successful step $a \mapsto b$. So it holds exactly when c is reached from a by t successful applications of `step`, and a trace of length 0 to a different configuration is impossible. Since `step` is a function, the run is deterministic.

Space monitoring (lines 147–189). `LocatedCfg` (lines 149–153) pairs a configuration with, per tape, the head's current position and the least and greatest positions seen so far, as integers; `locatedInit` starts every head at 0 with the input tape's range covering the input (lines 155–159); `headDisplacement` is -1, $+1$ or 0 for a move left, a move right or a write (lines 161–164); `monitor` (lines 166–182) applies the same table lookup to update the position and the running minimum and maximum of the scanned tape, and nothing else; `locatedStep` is `step` on the configuration followed by `monitor` (lines 184–186). `visitedSpace` (lines 188–189) is the sum over tapes of upper $-$ lower $+ 1$: the number of cells whose head position was ever visited. `countWord a` is the binary expansion of a, least significant bit first, with `[false]` for 0 (line 147), and `toNat` (lines 7–9) decodes such a word.

The exponent and the bound (lines 130–141).

```
runtimeScale n = n * (Real.log (Real.log (n + 16)))^2 /
Real.sqrt (Real.log (n + 16))
```

(lines 132–134, transcribed; the casts to $\mathbb{R}$ omitted), that is

$$\rho(n) \;=\; \frac{n\,\bigl(\ln\ln(n+16)\bigr)^2}{\sqrt{\ln(n+16)}},$$

with `Real.log` the natural logarithm. The manuscript fixes the same base: "Throughout, log denotes the natural logarithm" (p. 1, lines 48–49; again at p. 7, lines 326–327). The shift by 16 keeps $\ln\ln$ defined and positive at every $n \geq 0$ ($\ln 16 = 2.77$), so the theorem can be stated for all n with no proviso. Then

```
runtimeBound C A d L n = C * (L + 1)^d * 2^(A * runtimeScale n)
```

(lines 136–139, transcribed), a real number, with the real-exponent power. We write $\rho(n)$ for `runtimeScale n` throughout the paper.

2.2 The theorem, and the order of its quantifiers

Lines 194–217 state, for every k with $2 \le k$:

$\exists$ `machine`, `countTape`, `answerTape`, $C, A, S \in \mathbb{R}$, $d, e \in \mathbb{N}$,
$0 < C \ \wedge\ 0 < A \ \wedge\ 0 < S \ \wedge\ 0 < d \ \wedge\ 0 < e \ \wedge$
$\forall\, n \in \mathbb{N},\ \forall\, F : \texttt{CNF}\ (\texttt{Fin}\ n),\ \texttt{WidthAtMost}\ k\ F \ \rightarrow$
$\exists$ `time` $\in \mathbb{N}$, `final` : `LocatedCfg machine`,
`Trace` (`locatedStep machine`) `time` (`locatedInit machine` (`encodeCNF` F)) `final`
(lines 201–202)
$\wedge$ the count tape holds `countWord (count F)` (203–204)
$\wedge$ the answer tape holds the bit $[0 <$ `count` $F]$ (205–206)
$\wedge$ the halting state carries that bit (207)
$\wedge$ `toNat` (`countWord` (`count` F)) $=$ `count` F (208)
$\wedge$ $|$`countWord` (`count` F)$| \le n + 1$ (209)
$\wedge$ `time` $\le C\,(L+1)^d\, 2^{A\,\rho(n)}$, $L =$ `formulaBitLength` F (210–211)
$\wedge\ \forall$ prefix of the trace, `visitedSpace` $\le S\,(L+1)^e$ (212–217).

Three features of the order matter.

The machine and the five constants come after k and before n and F. They may depend on k and on nothing else; inside the universal quantifier only `time` and `final` are chosen. So for each k one fixed finite machine serves every formula of every size, and C, A, S, d, e are the same for all of them: the standard $O_k(1)$ reading. The table's size is $O_k(1)$ in the same sense, with no explicit function of k.

There is no lower bound on n. The manuscript's Theorem 1.1 and Proposition 6.4 are stated for $n \ge 20$ (p. 2, lines 56–57; p. 43, lines 2474–2475), and the solution's auxiliary `exact_count_eth` keeps $20 \le n$ (`SqrtLogSpace/Final.lean`, lines 104–109); the exported theorem does not. The $+16$ in $\rho(n)$ is what makes that possible.

The constants are existential. The statement asserts that values of C, A, S, d, e exist; it names none. The proof obtains them from existential lemmas and never evaluates them: at the point where the large-input run is assembled, the exponent constant is set to $A = 256 + A_t$ with $A_t \ge 1$ taken from a leaf contract (`SqrtLogSpace/NativeLargeEntry.lean`, line 51; `SqrtLogSpace/NativeLeafContract.lean`, lines 792–793), and the degree and the polynomial constants are built the same way. So the proof as written has $A \ge 257$, and no closed value of any of the five. The witness machine is explicit in one respect: it is compiled from a program over 80 one-bit registers and 1,220 tapes, with `countTape` $= 256$ and `answerTape` $= 1216$ (`SqrtLogSpace/Final.lean`, lines 98–101); its state count is a finite type's cardinality that nothing computes. Section 4 is the inspection of the proof; this section only reads what the statement fixes and what it leaves open.

2.3 The exponent, in numbers

Everything the title says rests on one function. With $y = \ln(n + 16)$,

$$\frac{\rho(n)}{n} \;=\; \frac{(\ln y)^2}{\sqrt{y}},$$

which rises from 0.64 at $n = 1$, crosses 1 between $n = 49$ and $n = 50$, peaks at $16/e^2 = 2.165$ where $\ln y = 4$, i.e. $y = e^4$, $n = e^{e^4} - 16 \approx 10^{23.7}$, and returns below 1 only where $(\ln y)^2 = \sqrt{y}$ again, at $y = 5503.7$, i.e. $n \approx e^{5504} \approx 10^{2390}$. Table 2.1 gives the values and Figures 2.1 and 2.2 draw them; every number is printed by `paper3/scripts/runtime\_scale.py` (run under `python3 -I`; it imports nothing from the clone and computes in y, so that no n beyond floating-point range is ever formed). The crossing at 50 is a fact about integers: $\rho(49)/49 = 0.9994$ and $\rho(50)/50 = 1.0027$. In the limit $\rho(n)/n \to 0$, since $(\ln y)^2/\sqrt{y} \to 0$: the exponent is $o(n)$, and that is a theorem in our companion Lean (Appendix A, A.2). Both facts are true at once, and the second is the one the title is about: the function is $o(n)$, and it is above n on every size anyone will ever write down.

Table 2.1: The verified exponent relative to n. $\rho(n) = n(\ln\ln(n+16))^2/\sqrt{\ln(n+16)}$, the comparator's `runtimeScale` (lines 132–134), natural logarithms. The verified bound is $C(L+1)^d\, 2^{A\rho(n)}$; at $A = 1$ its exponential factor is $2^{(\rho(n)/n)\cdot n}$, against 2^n for trying every assignment. Regenerate: `python3 -I paper3/scripts/runtime_scale.py`.

n	$\rho(n)/n$	$\rho(n)$
1	0.6443	0.64
20	0.8606	17.21
49	0.9994	48.97
50	1.0027	50.14
100	1.1146	111.46
125	1.1495	143.69
10^3	1.4229	1422.88
10^6	1.8550	1.85×10^6
10^{12}	2.0956	2.10×10^{12}
$10^{23.7}$	2.1654	—
10^{100}	1.9497	—
10^{1000}	1.2490	—
$e^{5504} \approx 10^{2390}$	1.0000	—
10^{3000}	0.9403	—

2.4 The title's claim, made precise

The title says the verified algorithm is slower than trying all 2^n assignments at every practical size. Read against the statement, that is the following, and nothing more.

1. **What is verified is a guarantee, not a running time.** The theorem says the machine halts within $C(L+1)^d\, 2^{A\rho(n)}$ steps. It does not say how many steps the machine takes on any input, and no implementation of the machine exists to be timed. A bound above 2^n does not say the machine is slow; it says the proof promises nothing better than 2^n there.

2. **The constants C, A, S, d, e are existential and never computed.** The comparator asserts their existence; the proof builds them from existential lemmas (above, with $A \geq 257$ as constructed) and states no value. The machine's state count is likewise never computed.

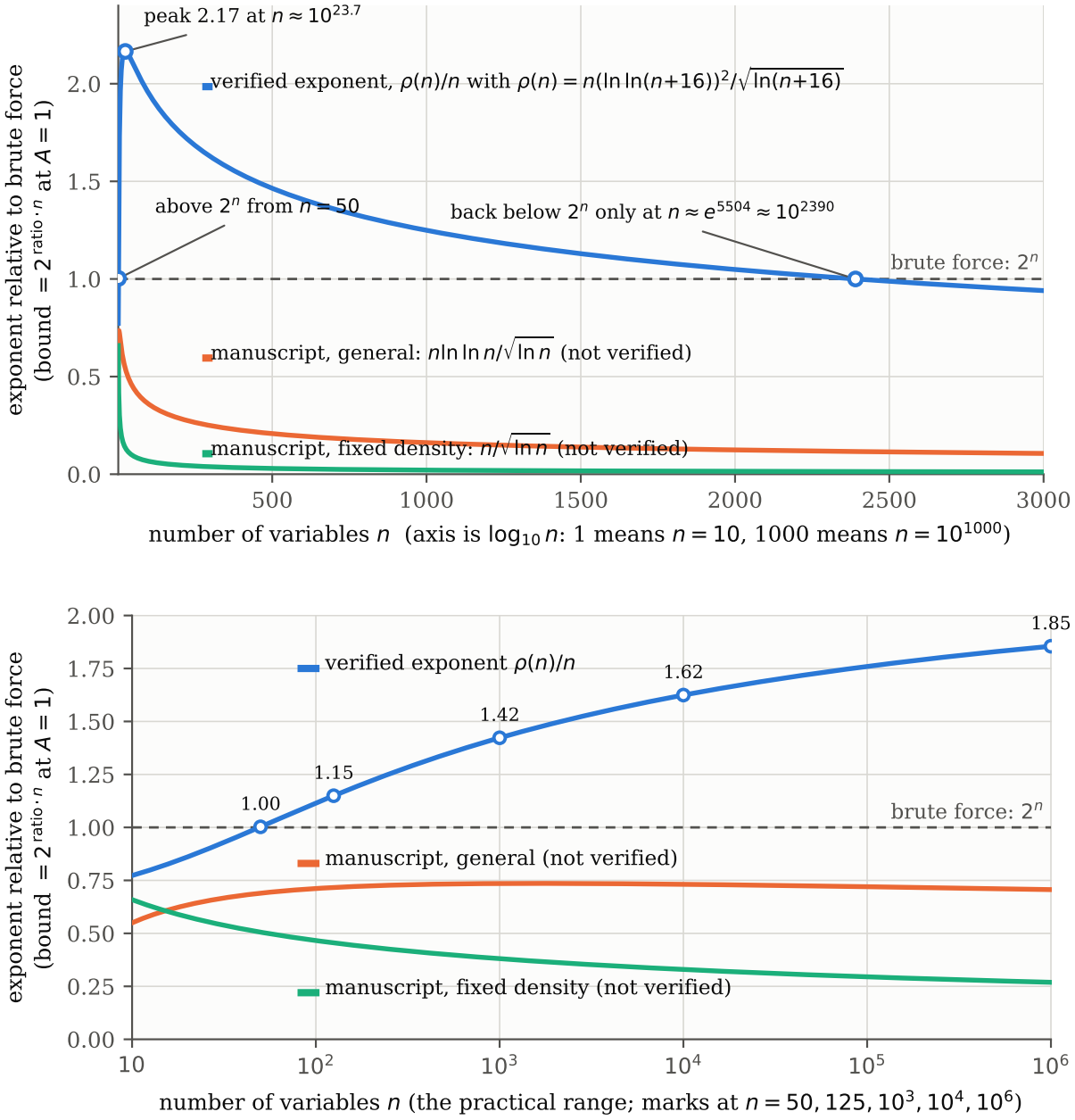


Figure 2.1: The verified exponent against brute force. Top: $\rho(n)/n$ over the whole range, the axis in $\log_{10} n$ from $n = 10$ to $n = 10^{3000}$; the dashed level is 2^n; beneath it the manuscript's two sharper exponents, $n \ln\ln n/\sqrt{\ln n}$ (Theorem 1.1, general) and $n/\sqrt{\ln n}$ (fixed clause density), which are not in the Lean library (Section 3). Bottom: the same three curves over the practical range $n = 10$ to 10^6, with $\rho(n)/n$ marked at $n = 50, 125, 10^3, 10^4, 10^6$. All curves at the constant $A = 1$; the verified A is unknown. Regenerate: `python3 -I paper3/scripts/rho_over_n.py`.

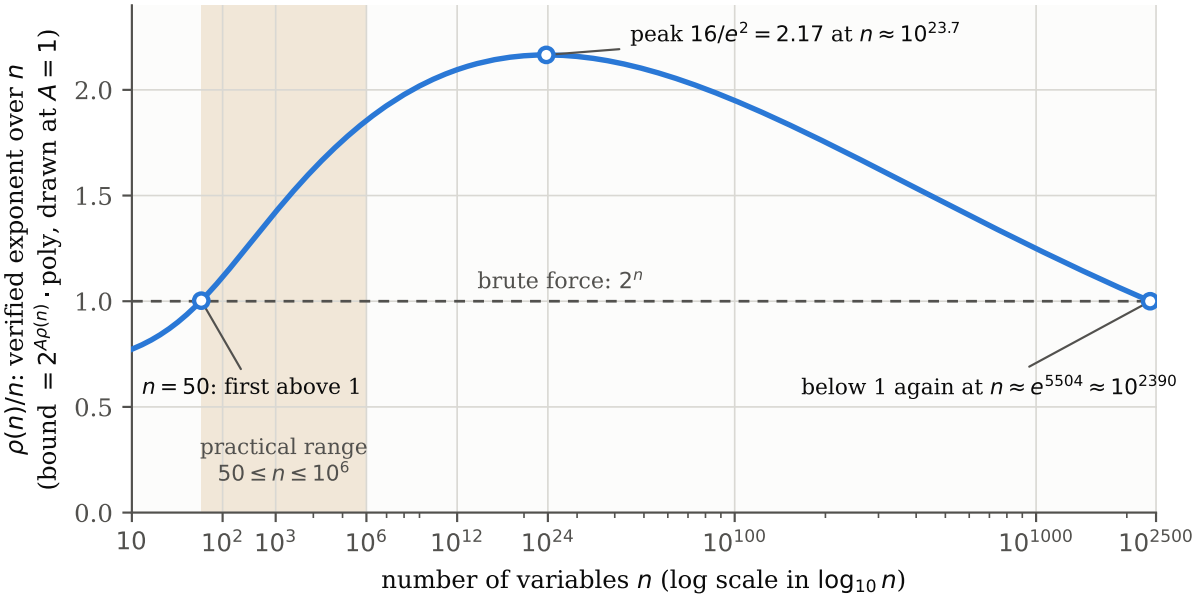


Figure 2.2: $\rho(n)/n$ alone, on a logarithmic axis in $\log_{10} n$ from $n = 10$ to $n = 10^{2500}$, with the level 1 (brute force, 2^n), the first integer above the level ($n = 50$), the peak ($16/e^2 = 2.17$ at $\ln(n+16) = e^4$, $n \approx 10^{23.7}$), the return below the level ($n \approx e^{5504} \approx 10^{2390}$), and the practical range $50 \le n \le 10^6$ shaded. Regenerate: `python3 -I paper3/scripts/runtime_scale.py`.

3. **With $A = 1$, the verified exponent exceeds n for every $50 \le n < e^{5504}$.** That is Table 2.1: the exponential factor of the bound is at least 2^n from 50 variables up to about 10^{2390}, and $2^{2.17\,n}$ at its worst. With the true A, whatever it is, the factor is $2^{A\rho(n)} \ge 2^{\rho(n)}$ once $A \ge 1$, and the proof's own construction has $A \ge 257$; so nothing is known that would put the bound below 2^n at any practical size.

4. **Hence, at every practical size, the verified guarantee is weaker than the guarantee of trying all assignments.** Trying all 2^n assignments and checking each against the m clauses counts the models in $2^n \cdot \mathrm{poly}(n+m)$ steps on any reasonable machine, with constants one can write down. The verified guarantee is $C(L+1)^d\, 2^{A\rho(n)}$ with $\rho(n) > n$ on $50 \le n < e^{5504}$ and C, A, d unknown. For every $C \ge 1$, $A \ge 1$ and $d \ge 1$ the verified bound is above 2^n on every n in that range, and the only values on record are the proof's, which has $1 \le C$ and $1 \le A$ in the large-input run's own statement (`SqrtLogSpace/NativeLargeEntry.lean`, line 12) and $A \ge 257$ in its construction. A proof with $A < 1$ would be a different proof; none is on record.

None of this is a defect of the formalisation or of the proof; it is what an $O_k(1)$ statement with a $2^{o(n)}$ exponent of this shape means when the exponent's constants are not tracked. The manuscript itself calls $\rho(n)$ "the weaker exponent" (p. 2, line 63), and its sharper exponents are not in the Lean library (Section 3). One more detail of base: the solution's internal ρ (`SqrtLogSpace/Asymptotics.lean`, lines 10–11) is in base 2 and is related to `runtimeScale` by `runtimeScale` $n \le 4\,\rho_2(n)$ for $n \ge 20$ (lines 189–190); the base changes the exponent by a constant factor that A absorbs, so nothing above depends on it.

2.5 What the statement does establish

For each fixed $k \geq 2$, a deterministic machine that, on the canonical encoding of any k-CNF on n variables with m clauses, halts with the exact number of satisfying assignments in binary on a named tape, within $C(L+1)^d\, 2^{A\rho(n)}$ steps, where $L = O(k\,n\,m)$ is polynomial in $n+m$ and $\rho(n) = o(n)$; and, because a `Finset` of clauses of width at most k has at most $\sum_{j \leq k} \binom{2n}{j} = O(n^k)$ members (2,604,376 at $n = 125$, $k = 3$), the whole bound is $2^{o(n)}$ in n alone. That is a deterministic $2^{o(n)} \cdot \mathrm{poly}(n+m)$ algorithm for #k-SAT in the standard sense, and therefore for k-SAT; it refutes the Exponential Time Hypothesis (Impagliazzo and Paturi, 2001) as stated. The statement reads as the manuscript's verified form, the weaker exponent of its Proposition 6.4 (p. 43, lines 2474–2476), without the $n \geq 20$ proviso. We found no field, definition or quantifier through which the statement could be kernel-true and mean something weaker than this; the audit it summarises is `reports/scratch/eth_audit/statement\_audit.md`. Our own restatement of the statement as a Lean `Prop`, character for character over a verbatim copy of the machine model, is (Appendix A, A.1); it is never proved and never an axiom there (Section 8).

2.6 What this does not claim

- **No running time of any machine.** Nothing here measures or estimates how long the verified machine, or any program, takes on any input. The title compares two *guarantees*; Section 7 is the only place in the paper where anything is timed, and it times ordinary solvers, not this machine.
- **No statement about the proof.** That the statement means what we say it means is a reading of 220 lines; whether the proof of it is sound is Section 4 and Section 5, which found no flaw. The one proof-side fact quoted here, $A = 256 + A_t$, is a line of the solution read for the value it assigns, not a claim about the argument.
- **No value of the constants.** $A \geq 257$ is what the proof writes down, not what the machine needs; a different proof of the same statement could have any $A > 0$. The statement itself fixes nothing.
- **Nothing about the manuscript's sharper exponents**, which are not in the Lean library and are Section 3's subject.
- **No claim about sizes beyond e^{5504}.** Past $n \approx 10^{2390}$ the verified factor is below 2^n at $A = 1$; we say nothing about that range except that it is not a practical one.

3 What the manuscript claims beyond the verified statement

Placeholder. Item 03 drafts this section: Theorem 1.1's two sharper exponents, Corollary 1.4's quantitative circuit bound, Appendix C's consequences, Appendix D, each tagged verified or paper-only with page and line of the extract. Ends with what this does not claim".

4 The proof, inspected

Placeholder. Item 04 drafts this section: the closure size, the zero counts, the 33 identical definitions, the compile lemmas concluding in the comparator's own `Trace`, the unpatched Mathlib, the absence of a recorded axiom audit, of CI and of a second kernel; joined with the replay of Section 5. Ends with what this does not claim".

5 Independent replay

The repository holds no recorded `#print axioms` output for the family, no continuous integration that builds its Lean, and `enable_nanoda: false` in the comparator's JSON (Section 4). We therefore built the solution's import closure ourselves, in isolation, at the toolchain the repository pins, and recorded what the kernel reports. This section says exactly what was run, where, and what came out; the scripts are quoted verbatim in `paper3/replay/README.md`, and the logs are under `paper3/replay/logs/`.

5.1 What was run, and where

A fresh copy of the clone (commit `4eaee875`) was built inside a `bwrap` (bubblewrap) sandbox on the project's machine: the system directories bound read-only, a private `tmpfs` home with nothing of the real home or of this repository visible, separate PID, IPC and UTS namespaces, the whole sandbox confined to eight cores (`taskset -c 0-7`) at the lowest scheduling priority, and the network on, because the repository's `lakefile` clones its 41 dependencies at build time and the Mathlib object files come from the Mathlib cache. Inside, `elan` was installed into the private home and fetched the pinned toolchain from the repository's own `lean-toolchain` file; then, in the clone's `lean/` directory, `lake exe cache get`, `lake build OAI.Algorithms.SubexponentialSAT.Main` (the solution's closure, not the collection's default target, which builds all 721 families), and `#print axioms` of the one theorem the comparator names, `OAI.Algorithms.SubexponentialSAT.SqrtLogSpace.exact_count_and_sat`, through `lake env lean`. Only text left the sandbox; nothing built inside it was executed outside it.

Two runs were needed. The first, started at 01:03 on 2026-11-05, failed at once at the build step because its command line carried `-j 8`, an option this version of Lake does not have (the whole of its build log is the one line `error: unknown short option '-j'`); nothing of the family was compiled. The second, started at 01:16 with that option removed and the core cap moved to `taskset`, is the run reported below. Both scripts, both versions, and the launch command are in the README; the first run's logs are kept beside the second's.

5.2 What came out

The build finished with exit code 0 in 33 minutes of wall clock on eight cores (01:16:25 to 01:49:31), compiling **3,091** modules with the prefix `OAI.Algorithms.SubexponentialSAT`—exactly the import closure the inspection of Section 4 enumerated, the same 3,091 names with an empty set difference both ways (`paper3/replay/counts.sh`; Lake's own job total, `12014`, counts the cached Mathlib and dependency modules, none of which was compiled). The log has no line containing `error` and none containing `sorry`; its eleven `warning` lines are Lake noting that

the lakefile's compatibility patches leave eleven dependencies with "local changes", and none of the eleven is Mathlib. Seventeen source files of the closure themselves carry `#print axioms` commands, so the build log records nineteen axiom lines as it goes, every one `[propext, Classical.choice, Quot.sound]`; among them `SqrtLogSpace/Final.lean`, lines 197–201, prints the axioms of `compiled_exact_count_and_sat`, `exact_count_and_sat`, `exact_count_eth`, `sat_polynomial_space` and `original_sat_main`. The separate `#print axioms` run after the build gives, for the comparator's theorem,

```
'OAI.Algorithms.SubexponentialSAT.SqrtLogSpace.exact_count_and_sat'
depends on axioms:  [propext, Classical.choice, Quot.sound]
```

(`paper3/replay/logs/run2/axioms.txt`). The versions are `Lean (version 4.34.1, commit 5045d005..., Release)`, `Lake version 5.0.0-src+5045d00`, and Mathlib at `d13f23b723b8a846827` from the clone's own manifest, with its object files from the Mathlib cache (8,908 files). The replay copy is at commit `4eaee875`, the same as the pristine clone, and the comparator file and the solution's files hash identically in both (`paper3/replay/logs/run2/hashes.txt`; the comparator statement's SHA-256 begins `4183b205`). The full build log is kept out of the repository by size; its SHA-256 (`d313f594...`) and its last 200 lines are committed.

So the sentence every later section relies on reads: *replayed*—the solution's closure builds from source at the pinned toolchain with upstream Mathlib, and the kernel reports the three standard axioms and nothing else for the theorem the comparator names. This moves the first three items of the inspection's trust boundary (Section 4): the build is no longer OpenAI's unpublished one, the `#print axioms` output is now on record, and the Mathlib revision was fetched from upstream and not patched.

5.3 The second kernel

The comparator's JSON for the family has `enable_nanoda: false`: OpenAI's check of this family ran the Lean kernel only. We ran a second kernel, in the same sandbox, after the build (`paper3/replay/nanoda.sh`, recorded in the README before it ran): `lean4export` (tag `v4.34.0`, commit `076e8e57`, built with the pinned `v4.34.1` toolchain so that it reads the object files the build produced) exported `exact_count_and_sat` and its transitive dependencies from the built environment—2,355,666,449 bytes of NDJSON, 43,633,742 lines, format version 3.1.0, Lean githash `5045d005...`, in 68 seconds—and `nanoda_lib` (nanoda contributors, 2026) (version 0.4.31, commit `8de7e0c4`, 2026-10-29; an external type checker for Lean 4 written in Rust, which shares no code with the Lean kernel) checked that export with `propext`, `Classical.choice` and `Quot.sound` as the permitted axioms and the `Nat` and `String` literal extensions on. Its output (`paper3/replay/logs/nanoda/nanoda_check.txt`):

```
Checked 95176 declarations with no errors
```

in 110 seconds on eight cores, exit code 0, after printing back the theorem's statement and the three axioms from its own parse of the export. The export contains exactly three `axiom` declarations (`grep -c '"axiom"'` on the file), so nothing else could have been permitted. The export file (SHA-256 `bdbc9afe...`, `paper3/replay/logs/nanoda/export_sha256.txt`) stays in the sandbox directory by size; the configuration, the build logs and the checker's output are committed.

5.4 What this does not claim

- The replay trusts the Lean kernel of the pinned toolchain (`v4.34.1`) and upstream Mathlib at the pinned revision; it moves the proof's trust from "OpenAI's unpublished build" to "a build we ran", not from the Lean kernel to anything else. The second kernel removes the Lean kernel from the list for the theorem's dependencies as exported, but adds `lean4export` (that it exports what the environment holds) and `nanoda_lib` (that it checks what it parses) to it; two independent checkers agreeing is evidence, not a proof that either is correct.
- It is not the comparator run. The comparator checks that the solution's exported statement matches the challenge's by `lean4export`; we did not run that tool chain, and the statement-level match the paper relies on is the textual one of Section 4 (the 33 definitions and the theorem statement, identical character for character after whitespace normalisation).
- The counts above are counts of lines in one build's log on one machine with the named versions; a different Lake would print different job totals (the 12,014 includes the cached Mathlib modules), and a different machine a different time.
- Nothing here bears on what the theorem *says* (Section 2) or on the manuscript's unverified exponents (Section 3); a replayed proof of a weak bound is a replayed proof of a weak bound.

6 What follows

Placeholder. Item 05 drafts this section: which hypotheses the verified statement refutes (ETH, SETH, Gap-ETH, #ETH, NSETH, the randomised forms) and how; in general terms, that a lower bound derived from ETH or SETH loses its premise while a statement "no $(c-\varepsilon)^k$ algorithm" is not itself refuted (the manuscript's §1.1.3); no example from any width parameter. Ends with what this does not claim".

7 For the practitioner

Placeholder. Item 06 drafts this section: a SAT solver and a model counter on random 3-CNF at $n = 20 \ldots 60$ against $2^{\rho(n)}$ and 2^n (figure labelled `fig:practitioner`, from `paper3/scripts/practitioner.py`); the verified guarantee is never the binding one on any machine. Ends with what this does not claim".

8 The statement in our Lean

Placeholder. Item 07 drafts this section: the restatement `SubexpCount` over the verbatim machine model under `lean/MOSPFormalization/ETH/`, the axiom audit, the byte-level rediff against the comparator file; no theorem of ours about the hypothesis is proved, and none is claimed. Ends with what this does not claim".

9 Related work and context

Placeholder. Item 08 drafts this section: the human audit of the August 2026 results (arXiv 2608.14673), the working group named in the collection's history, Impagliazzo–Paturi, Impagliazzo–Paturi–Zane, the comparator mechanism, nanoda; "we found no prior statement", never "first". Ends with what this does not claim".

10 Conclusion

Placeholder. Item 09 drafts this section: what is claimed and what is not; every entry of `paper3/ETH\_AMBIGUOUS.MD` that affects the text, listed as open by number. Ends with what this does not claim".

Data, code and proofs

Placeholder. Item 11 completes this statement: the repository, the scripts under `paper3/scripts/` that regenerate every number, the replay's commands and log hashes under `paper3/replay/`, the Lean names and the axiom audit under `paper3/lean/`, and the read-only clone of `https://github.com/openai/math` at commit `4eaee875`.

Acknowledgements

Placeholder. Item 11 writes the acknowledgements, with the disclosure of AI use in the form of `paper1/latex/sec8\_ack.tex`.

References

Russell Impagliazzo and Ramamohan Paturi. On the complexity of k-SAT. *Journal of Computer and System Sciences*, 62(2):367–375, 2001. doi: 10.1006/jcss.2000.1727.

nanoda contributors. nanoda: a type checker for Lean 4 written in Rust. `https://github.com/ammkrn/nanoda_lib`, 2026. Repository README; author, version and date recorded by item 08.

OpenAI. The Exponential Time Hypothesis is False. OpenAI Math Release preprint OAI:The-Exponential-Time-Hypothesis-is-False-October-25-2026, 2026.

Appendix A Formal names

Every Lean name the paper cites, with its file under `lean/MOSPFormalization/ETH/`; the axiom audit is `paper3/lean/axiom_check.lean` with its committed log.

	result	Lean names
A.1	The comparator statement restated as a `Prop`, `SubexpCount` k, over the verbatim copy of the machine model; never proved, never an axiom	`MOSPFormalization.SubexpPathwidth.SubexpCount`
A.2	$\rho(n) = o(n)$: the verified exponent is subexponential in the limit	`MOSPFormalization.SubexpPathwidth.runtimeScale\_isLittleO`

Placeholder. Item 07 moves these declarations to `lean/MOSPFormalization/ETH/` and rewrites the names above to their new namespace.